

Machine Learning II

Lecture 4: Matrix Factorization and Latent Factors

Alexander Quispe
ENEI – INEI · PEU-CD 2026

October 16, 2026

Outline

Recap and Motivation

Matrix Norms

Latent Factor Models

Multitask Learning and Bilinear Models

Non-Negative Matrix Factorization

Bridge to Deep Learning

Summary and Next Steps

Recap & Motivation

From sequences to structure

Where We've Been

Lectures 1–5 covered *supervised* learning with deep models:

- Lecture 1: perceptrons, the universal approximation theorem.
- Lecture 2: backpropagation.
- Lecture 3: regularization, SGD.
- Lecture 4: convolutional networks.
- Lecture 5: recurrent networks (RNN, LSTM, GRU).

Common pattern: we always had labeled pairs $(\mathbf{x}_i, y_i)$ and minimized a prediction loss.

Today we shift gears: what if we have only the matrix $\mathbf{Y}$, with no separate features and many missing entries?

Three Motivating Problems

1. **Recommendation:** a matrix of user-by-item ratings, mostly empty. Predict the missing entries.
2. **Causal panels:** a matrix of unit-by-time outcomes, where treated units have unobserved counterfactuals. Recover what would have happened without treatment.
3. **Topic discovery:** a document-by-word count matrix. Find a small set of topics that explain the documents.

Today's unifying view

All three are **matrix factorization** problems:

$$\mathbf{Y} \approx \mathbf{U}\mathbf{V}^T$$

with $\mathbf{U}$ low-rank rows and $\mathbf{V}$ low-rank columns.

Why a Deep Learning Class Cares

Three reasons to spend a class on this:

1. **Embedding layers are matrix factorization.** `nn.Embedding(N, K)` stores one row per entity. The dot product of two rows is exactly $\mathbf{u}_i^\top \mathbf{v}_j$.
2. **PCA is a linear autoencoder.** Replacing the linear bottleneck with a neural network gives you a deep autoencoder.
3. **Word2vec is implicit matrix factorization.** The skip-gram with negative-sampling objective factorizes a shifted PMI matrix (Levy & Goldberg, 2014).

Latent factor models are the linear ancestor of every modern embedding-based model.

Today's Roadmap (Annotated)

1. **Linear algebra recap.** Orthogonal matrices, SVD, PCA – the foundation.
2. **Matrix norms.** Frobenius and trace norms; the convex relaxation of rank.
3. **Latent factor models.** Collaborative filtering, missing values, alternating optimization.
Reframed as panel data with missing potential outcomes.
4. **Multitask learning.** Latent factors as shared low-dimensional features across tasks.
5. **Non-negative matrix factorization.** Parts-based representations, topic modeling.
6. **Bridge to deep learning.** Embedding layers, autoencoders, word2vec.

What We Carry Over From Machine Learning I

Lecture 6 of ML I proved, in full, the linear-algebra facts this lecture builds on. They are used, not re-derived:

- **Projections.** $\mathbf{P} = \mathbf{V}\mathbf{V}^\top$ with $\mathbf{V}^\top \mathbf{V} = \mathbf{I}_k$ is symmetric and idempotent; $\|\mathbf{x}\|^2 = \|\mathbf{P}\mathbf{x}\|^2 + \|(\mathbf{I} - \mathbf{P})\mathbf{x}\|^2$.
- **PCA.** The top eigenvector of $\mathbf{S} = \frac{1}{N}\mathbf{X}^\top \mathbf{X}$ maximizes projected variance (Lagrangian); the top k maximize $\text{tr}(\mathbf{V}^\top \mathbf{S} \mathbf{V})$ (Ky Fan); retained variance $= \lambda_1 + \dots + \lambda_k$; maximum variance $\Leftrightarrow$ minimum reconstruction error.
- **SVD.** For centred $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^\top$, the right singular vectors are the principal components and $\lambda_j = d_j^2 / N$.
- **Eckart–Young.** Among all matrices of rank $\leq k$, the truncated SVD $\mathbf{U}_k \mathbf{D}_k \mathbf{V}_k^\top$ is closest to $\mathbf{X}$ in Frobenius norm, with error $\sum_{j>k} d_j^2$.

Today's question

Eckart–Young needs every entry of $\mathbf{X}$. What if most entries are missing — a user has rated 20 of 10,000 films, a firm is observed only before a policy? That is the latent-factor problem, and the trace norm is how the rank constraint survives it.

Matrix Norms

Frobenius, trace, and the rank surrogate

The Frobenius Norm

Definition:

$$\|\mathbf{X}\|_{\text{Fro}} = \sqrt{\sum_{i,j} x_{ij}^2}$$

the matrix version of the L2 vector norm.

In terms of singular values:

$$\|\mathbf{X}\|_{\text{Fro}}^2 = \sum_d \sigma_d^2$$

Why? Use the trace identity $\|\mathbf{X}\|_{\text{Fro}}^2 = \text{trace}(\mathbf{X}^\top \mathbf{X})$ and substitute SVD:

$$\text{trace}(\mathbf{X}^\top \mathbf{X}) = \text{trace}(\mathbf{V} \mathbf{\Sigma}^2 \mathbf{V}^\top) = \text{trace}(\mathbf{\Sigma}^2) = \sum_d \sigma_d^2.$$

Useful for L2-style regularization of matrix-valued parameters.

The Rank Norm and the Trace Norm

Rank norm (count of nonzero singular values):

$$\|\mathbf{X}\|_{\text{Rank}} = \sum_d \mathbf{1}[\sigma_d > 0]$$

Problem: this is discrete and discontinuous – impossible to optimize with gradient methods.

Trace norm (a.k.a. nuclear norm):

$$\|\mathbf{X}\|_* = \sum_d \sigma_d = \text{trace}\left(\sqrt{\mathbf{X}^\top \mathbf{X}}\right)$$

Intuition: this is the L1 norm *of the singular values*.

The trace norm is the convex envelope of the rank norm – the same way L1 is the convex relaxation of L0.

Convex Relaxation: A Familiar Pattern

For vectors:

Goal	Discrete penalty	Convex surrogate
sparse vector	$\ \mathbf{w}\ _0$ (count of nonzeros)	$\ \mathbf{w}\ _1$

⇒ Lasso regression.

For matrices:

Goal	Discrete penalty	Convex surrogate
low-rank matrix	$\text{rank}(\mathbf{X})$	$\ \mathbf{X}\ _*$ (trace norm)

⇒ Trace-norm regularized matrix factorization.

Same idea, different object. Both replace a non-convex sparsity penalty with its tightest convex lower bound.

Useful Properties of Matrix Norms

Cauchy–Schwarz:

$$\langle \mathbf{A}, \mathbf{B} \rangle^2 = \text{trace}(\mathbf{A}^\top \mathbf{B})^2 \leq \|\mathbf{A}\|_{\text{Fro}}^2 \cdot \|\mathbf{B}\|_{\text{Fro}}^2$$

AM–GM inequality:

$$\|\mathbf{A}\| \cdot \|\mathbf{B}\| \leq \frac{1}{2}(\|\mathbf{A}\|^2 + \|\mathbf{B}\|^2)$$

Orthogonal invariance: for full-rank orthogonal $\mathbf{U}$,

$$\|\mathbf{UA}\|_{\text{Fro}} = \|\mathbf{A}\|_{\text{Fro}}, \quad \|\mathbf{UA}\|_* = \|\mathbf{A}\|_*.$$

Both norms see only singular values, not the choice of basis.

Trace norm of a diagonal matrix: just the L1 of its diagonal entries:

$$\|\mathbf{A}\|_* = \sum_i |A_{ii}| \quad \text{if } \mathbf{A} \text{ is square diagonal.}$$

Latent Factor Models

Low-rank reasoning with missing data

Motivation: The Netflix Problem

Setup: an $M \times N$ matrix $\mathbf{Y}$ where Y_{ij} is the rating user i gave movie j .

Goal: predict the ratings users *didn't* provide.

$$\mathbf{Y} \approx \begin{matrix} \text{users} \times \text{movies} \\ \boxed{M \times N} \end{matrix} = \begin{matrix} \boxed{\mathbf{U}} \\ M \times K \end{matrix} \begin{matrix} \boxed{\mathbf{V}^T} \\ K \times N \end{matrix}$$

Model:

$$Y_{ij} \approx \mathbf{u}_i^T \mathbf{v}_j$$

where $\mathbf{u}_i$ is a K -dimensional “user vector” and $\mathbf{v}_j$ is a “movie vector”. The K latent factors encode taste and content.

If $\mathbf{Y}$ were complete, we'd just take its SVD. But it isn't.

Worked example: 4 users, 5 movies, $K = 2$

Observed ratings (-2 to $+2$ scale; $?$ = missing):

		Aveng.	Schind.	Sav.Ry.	Nott.Hill	Dune
$\mathbf{Y} =$	Alice	+1.6	-1.6	?	?	+0.6
	Bob	-1.1	?	+0.3	-0.1	?
	Carol	?	-0.4	+1.1	?	+1.0
	Dave	-0.2	?	?	+1.0	-0.9

$M=4$, $N=5$, 11 of 20 entries observed.

After fitting, one possible solution is

$$\mathbf{U} = \begin{pmatrix} +1.0 & +0.8 \\ -0.5 & -0.7 \\ +0.9 & -0.6 \\ -0.7 & +0.6 \end{pmatrix}_{4 \times 2} \quad \mathbf{V} = \begin{pmatrix} +1.0 & +0.8 \\ -1.0 & -0.8 \\ +0.6 & -0.9 \\ -0.8 & +0.7 \\ +0.9 & -0.4 \end{pmatrix}_{5 \times 2}$$

Rows of $\mathbf{U}$ = user vectors $\mathbf{u}_i$; rows of $\mathbf{V}$ = movie vectors $\mathbf{v}_j$. The two latent dimensions happen to align with *action-ness* (col. 1) and *lightness* (col. 2) — labels are read off *post hoc*.

Worked example: predictions $\hat{\mathbf{Y}} = \mathbf{UV}^\top$

Recipe: $\hat{Y}_{ij} = \mathbf{u}_i^\top \mathbf{v}_j$.

Observed entry — Alice $\times$ Avengers:

$$\mathbf{u}_{\text{Alice}}^\top \mathbf{v}_{\text{Aveng.}} = (1.0)(1.0) + (0.8)(0.8) = 1.00 + 0.64 = 1.64.$$

Observed: +1.6. ✓

Missing entry — Alice $\times$ Saving Ryan:

$$\mathbf{u}_{\text{Alice}}^\top \mathbf{v}_{\text{Sav.Ry.}} = (1.0)(0.6) + (0.8)(-0.9) = 0.60 - 0.72 = -0.12.$$

Action match cancels with tone mismatch $\Rightarrow$ Alice would skip it.

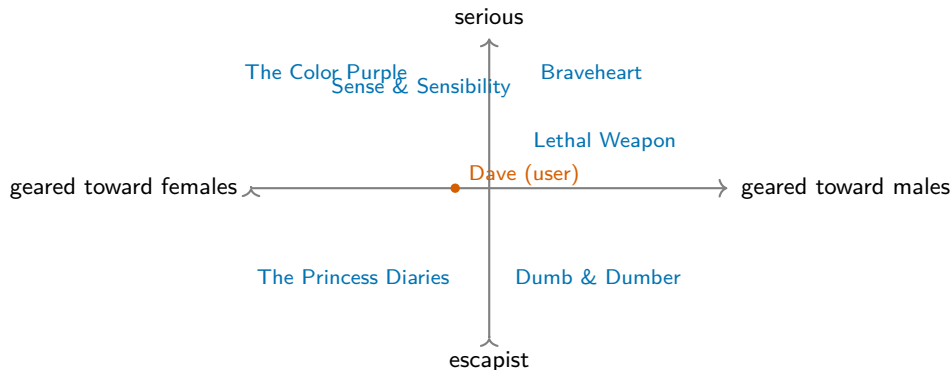
Full $\mathbf{UV}^\top$ (**bold** = was missing in $\mathbf{Y}$):

		Aveng.	Schind.	Sav.Ry.	Nott.Hill	Dune
$\hat{\mathbf{Y}} =$	Alice	+1.64	-1.64	- 0.12	- 0.24	+0.58
	Bob	-1.06	+ 1.06	+0.33	-0.09	- 0.17
	Carol	+ 0.42	-0.42	+1.08	- 1.14	+1.05
	Dave	-0.22	+ 0.22	- 0.96	+0.98	-0.87

Recommendations: Bob $\rightarrow$ *Schindler*; Carol $\rightarrow$ *Avengers*; Dave $\rightarrow$ *Notting Hill*.

Latent Factors: Interpretation

Hypothetical 2D factor space:



Reading the factors:

- Movie vector $\mathbf{v}_j$ = position in factor space (e.g., serious + male-skewed).
- User vector $\mathbf{u}_i$ = the user's preference direction.
- Predicted rating $\mathbf{u}_i^\top \mathbf{v}_j$ = how aligned the user's taste is with the movie.

Collaborative Filtering: The Real Problem

Reality: most of $\mathbf{Y}$ is missing.

- M users $\times$ N items, with M, N in the millions.
- Each user rates only a tiny fraction of items.
- No user features, no item features – only the observed ratings.

Goal: learn $\mathbf{U} \in \mathbb{R}^{M \times K}$ and $\mathbf{V} \in \mathbb{R}^{N \times K}$ such that $\mathbf{UV}^T$ reconstructs the observed entries *and* fills in the missing ones plausibly.

Assumption that makes this possible:

Low-rank assumption

There exists a low-rank subspace ($K \ll M, N$) that captures all the variation across users and items.

Latent Factor Formulation

Observed entries $S = \{(i, j) : Y_{ij} \text{ observed}\}$.

Let $\omega_{ij} = 1$ if $(i, j) \in S$, else 0.

Objective:

$$\arg \min_{\mathbf{U}, \mathbf{V}} \frac{\lambda}{2} (\|\mathbf{U}\|_{\text{Fro}}^2 + \|\mathbf{V}\|_{\text{Fro}}^2) + \sum_{i,j} \omega_{ij} (Y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j)^2$$

Two terms, doing different jobs:

- **Reconstruction loss**: fit observed ratings.
- **Frobenius regularizer**: shrink $\mathbf{U}, \mathbf{V}$ toward zero. Prevents overfitting and – as we'll see – caps the effective rank.

Hyperparameter: λ controls bias–variance tradeoff. Tune by held-out validation.

Connection to the Trace Norm

Claim: the Frobenius regularizer on $(\mathbf{U}, \mathbf{V})$ equals a trace-norm penalty on $\mathbf{W} = \mathbf{UV}^\top$.

Formally,

$$\|\mathbf{W}\|_* = \min_{\mathbf{W}=\mathbf{UV}^\top} \frac{1}{2} (\|\mathbf{U}\|_{\text{Fro}}^2 + \|\mathbf{V}\|_{\text{Fro}}^2).$$

Proof sketch (one direction): let $\mathbf{W} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ be the SVD, and choose $\mathbf{A} = \mathbf{U}\sqrt{\mathbf{\Sigma}}$, $\mathbf{B} = \mathbf{V}\sqrt{\mathbf{\Sigma}}$. Then $\mathbf{AB}^\top = \mathbf{W}$ and

$$\frac{1}{2} (\|\mathbf{A}\|_{\text{Fro}}^2 + \|\mathbf{B}\|_{\text{Fro}}^2) = \frac{1}{2} (\text{trace}(\mathbf{\Sigma}) + \text{trace}(\mathbf{\Sigma})) = \text{trace}(\mathbf{\Sigma}) = \|\mathbf{W}\|_*.$$

Why care? The Frobenius regularizer on the factors is *secretly* a low-rank inducer on the product. Same effect, but easier to optimize because the factors are explicit and small.

The Same Idea: Three Equivalent Views

View	Penalty
Rank-constrained	$\text{rank}(\mathbf{W}) \leq K$ (non-convex)
Trace-norm regularized	$\lambda \ \mathbf{W}\ _*$ (convex, but slow)
Factorized Frobenius	$\frac{\lambda}{2} (\ \mathbf{U}\ _{\text{Fro}}^2 + \ \mathbf{V}\ _{\text{Fro}}^2)$ with $\mathbf{W} = \mathbf{UV}^\top$ (non-convex but fast)

In practice: the third form wins. It's smooth, it has explicit factors we can store, and (for fixed K) it scales to massive matrices.

This is the form used by every recommender system, every embedding layer, and every modern matrix-completion econometrics method.

This Is Just Panel Data

Econometrics setup (e.g., Bai 2009, *Econometrica*):

$$Y_{it} = \lambda_i^\top \mathbf{F}_t + \varepsilon_{it}$$

where λ_i are unit-specific “loadings” and $\mathbf{F}_t$ are time-specific “factors”.

Stacking: let $\mathbf{\Lambda} \in \mathbb{R}^{N \times K}$ stack the loadings, $\mathbf{F} \in \mathbb{R}^{T \times K}$ stack the factors. Then

$$\mathbf{Y} = \mathbf{\Lambda} \mathbf{F}^\top + \varepsilon.$$

This is *exactly* the latent factor model with $\mathbf{U} = \mathbf{\Lambda}$ and $\mathbf{V} = \mathbf{F}$.

ML language	Econometrics language
user vectors $\mathbf{U}$	unit fixed effects (loadings) $\mathbf{\Lambda}$
item vectors $\mathbf{V}$	time fixed effects (factors) $\mathbf{F}$
low rank K	limited heterogeneity
missing entries	untreated / treated observations

Causal Panel Data: The Counterfactual

Setup: units i over time t . Treatment $D_{it} \in \{0, 1\}$.

We observe $Y_{it}(0)$ when $D_{it} = 0$, and $Y_{it}(1)$ when $D_{it} = 1$.

Causal effect of interest:

$$\tau_{it} = Y_{it}(1) - Y_{it}(0).$$

We never observe both. But we observe one, and the other is exactly a missing matrix entry.

Idea (Athey, Bai, Imbens, Khosravi, 2021, JASA):

1. Stack $Y(0)$ for all units and time periods into a matrix.
2. Mask the entries where $D_{it} = 1$ (missing counterfactual).
3. Run latent-factor matrix completion on the masked matrix.
4. Predicted entries are estimated counterfactuals; subtract from observed treated to get $\hat{\tau}_{it}$.

Same algorithm as Netflix – different application, profound implications.

Synthetic Control as a Special Case

Classical synthetic control (Abadie et al., 2010): the counterfactual of a treated unit is a weighted average of control units.

Matrix completion view:

- Restrict to one treated unit + many controls.
- Pre-treatment fit determines the unit's loading λ_i .
- Time factors $\mathbf{F}_t$ come from controls.
- Counterfactual $= \lambda_i^\top \mathbf{F}_t$.

Why matrix completion is more general:

- Handles *multiple* treated units.
- Handles staggered adoption (different units treated at different times).
- No simplex constraint on weights.
- Implements the same low-rank assumption with fewer restrictions.

Software: `gsynth` (R, Xu 2017) and `pysyncon` (Python).

Optimization: How Do We Actually Fit This?

Objective:

$$\mathcal{L}(\mathbf{U}, \mathbf{V}) = \frac{\lambda}{2} (\|\mathbf{U}\|_{\text{Fro}}^2 + \|\mathbf{V}\|_{\text{Fro}}^2) + \sum_{i,j} \omega_{ij} (Y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j)^2$$

Bad news: jointly non-convex in $(\mathbf{U}, \mathbf{V})$.

Good news: convex in $\mathbf{U}$ if $\mathbf{V}$ is fixed, and vice versa. This suggests two strategies.

Two algorithms:

1. **Stochastic gradient descent (SGD):** update one parameter at a time using a single (i, j) at a time.
2. **Alternating Least Squares (ALS):** fix $\mathbf{V}$, solve for $\mathbf{U}$ optimally; then fix $\mathbf{U}$, solve for $\mathbf{V}$.

ALS: Closed-Form Update

Fix $\mathbf{V}$ and ask: what is the optimal $\mathbf{u}_i$ for user i ?

Take $\partial \mathcal{L} / \partial \mathbf{u}_i = 0$:

$$\lambda \mathbf{u}_i - \sum_j \omega_{ij} \mathbf{v}_j (Y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j) = 0.$$

Rearranging:

$$\mathbf{u}_i = \left(\lambda \mathbf{I}_K + \sum_j \omega_{ij} \mathbf{v}_j \mathbf{v}_j^\top \right)^{-1} \left(\sum_j \omega_{ij} Y_{ij} \mathbf{v}_j \right)$$

This is just regularized linear regression of user i 's observed ratings on the (fixed) movie vectors.

ALS algorithm:

1. Initialize $\mathbf{U}, \mathbf{V}$ randomly.
2. Loop: for each i , update $\mathbf{u}_i$ using the formula above. Then for each j , update $\mathbf{v}_j$ symmetrically.
3. Until convergence.

SGD vs ALS: Tradeoffs

Aspect	ALS	SGD
Iterations to converge	few (10–50)	many (millions of steps)
Per-iteration cost	$O(K^3)$ matrix solve per row	$O(K)$ per observation
Parallelism	easy across rows	harder; per-step updates
Streaming data	no – needs full matrix	yes – one observation at a time
High dimensions (K large)	slow	fast

Practical advice:

- Small to medium K (≤ 50), data fits in memory $\rightarrow$ **ALS**.
- Massive data, streaming, or large $K \rightarrow$ **SGD** (or its mini-batch cousin).
- Modern deep models always use SGD-family methods (Adam, etc.) because they share the same optimizer with the rest of the network.

Section Summary

What we built:

- Latent factor model: $Y_{ij} \approx \mathbf{u}_i^\top \mathbf{v}_j$.
- Frobenius regularizer on $\mathbf{U}, \mathbf{V} \Leftrightarrow$ trace norm on $\mathbf{UV}^\top \Leftrightarrow$ low-rank prior.
- ALS (closed-form per row) and SGD (per observation).

Reframing for econ:

- Same math as panel data with interactive fixed effects.
- Matrix completion = causal inference with missing potential outcomes.
- Generalizes (and clarifies) synthetic control.

Up next: latent factors as a tool for *multitask learning* – many related prediction problems sharing a low-dimensional feature subspace.

Multitask Learning & Bilinear Models

One subspace, many tasks

The Multitask Setup

M **tasks**, each with its own dataset $S^m = \{(\mathbf{x}_i, y_i^m)\}$.

Naive approach: learn M independent linear models:

$$\arg \min_{\mathbf{w}_1, \dots, \mathbf{w}_M} \sum_m \left[\frac{\lambda}{2} \|\mathbf{w}_m\|^2 + \sum_i (y_i^m - \mathbf{w}_m^\top \mathbf{x}_i)^2 \right]$$

Concrete examples:

- Personalized recommendations: one task per user.
- Country-specific demand forecasts.
- Per-firm earnings prediction.

Problem with naive approach: $D \times M$ parameters, no sharing. Tasks with little data overfit.

If the M tasks are related, ignoring that fact wastes information.

Trace-Norm Regularization Across Tasks

Stack the per-task weight vectors:

$$\mathbf{W} = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_M] \in \mathbb{R}^{D \times M}$$

Key idea: encourage $\mathbf{W}$ to be *low rank*.

A low-rank $\mathbf{W}$ means task vectors live in a small shared subspace of $\mathbb{R}^D$ – exactly the structure we want when tasks are related.

Regularizer:

$$\arg \min_{\mathbf{W}} \frac{\lambda}{2} \|\mathbf{W}\|_* + \sum_m \sum_i (y_i^m - \mathbf{w}_m^\top \mathbf{x}_i)^2$$

Trace norm couples the tasks. It pulls them toward a common low-dimensional representation, sharing strength across tasks.

Bilinear Reformulation

Equivalent factored form:

$$\arg \min_{\mathbf{U}, \mathbf{V}} \frac{\lambda}{2} (\|\mathbf{U}\|_{\text{Fro}}^2 + \|\mathbf{V}\|_{\text{Fro}}^2) + \frac{1}{2} \sum_m \sum_i (y_i^m - \mathbf{u}_m^\top \mathbf{V} \mathbf{x}_i)^2$$

Reading the model:

- $\mathbf{V} \mathbf{x}_i \in \mathbb{R}^K$ is a **shared low-dimensional feature representation** of input $\mathbf{x}_i$. Same $\mathbf{V}$ for every task.
- $\mathbf{u}_m \in \mathbb{R}^K$ is a **small per-task model** living in this shared subspace.

Parameter counts:

- Independent: $D \cdot M$ (full per-task vectors).
- Bilinear: $D \cdot K + M \cdot K = K(D + M)$ – much smaller when K is small.

Statistically more efficient if the low-rank assumption is right.

Reduction: Bilinear Models $\supset$ Collaborative Filtering

Bilinear model:

$$\arg \min_{\mathbf{U}, \mathbf{V}} \frac{\lambda}{2} (\|\mathbf{U}\|_{\text{Fro}}^2 + \|\mathbf{V}\|_{\text{Fro}}^2) + \frac{1}{2} \sum_m \sum_i (y_i^m - \mathbf{u}_m^\top \mathbf{V} \mathbf{x}_i)^2$$

Special case: let $\mathbf{x}_i = \mathbf{e}_i$ (one-hot indicator for item i). Then $\mathbf{V} \mathbf{x}_i = \mathbf{v}_i$, the i -th column of $\mathbf{V}$, and the model reduces to

$$\arg \min_{\mathbf{U}, \mathbf{V}} \frac{\lambda}{2} (\|\mathbf{U}\|_{\text{Fro}}^2 + \|\mathbf{V}\|_{\text{Fro}}^2) + \frac{1}{2} \sum_m \sum_i (y_i^m - \mathbf{u}_m^\top \mathbf{v}_i)^2$$

This is exactly collaborative filtering.

Conclusion: latent factor models are a *special case* of bilinear multitask learning where the inputs are one-hot indicators.

With *real features* (not just indicators) we get a richer model: “cold-start” predictions for new users with side information.

General Bilinear Models

Most general form:

$$\arg \min_{\mathbf{U}, \mathbf{V}} \frac{\lambda}{2} (\|\mathbf{U}\|_{\text{Fro}}^2 + \|\mathbf{V}\|_{\text{Fro}}^2) + \frac{1}{2} \sum_m (y_i^m - \mathbf{z}_m^\top \mathbf{U}^\top \mathbf{V} \mathbf{x}_i)^2$$

Two side-information vectors:

- $\mathbf{z}_m$: features of task m (e.g., demographics of user m).
- $\mathbf{x}_i$: features of input i (e.g., text of news article i).

$\mathbf{U}$ projects the task features into the shared K -dim space; $\mathbf{V}$ projects the input features into the same space; the dot product is the score.

Common in econ applications:

- Trade flows: source country $\times$ destination country.
- Firm–worker matching.
- Demand: consumer attributes $\times$ product attributes.

Aside: Non-Linear Projections

So far: $\mathbf{V}\mathbf{x}_i$ is a *linear* projection from $\mathbb{R}^D$ into $\mathbb{R}^K$.

Replace $\mathbf{V}$ with a neural network:

$$\mathbf{V}\mathbf{x}_i \longrightarrow \phi_{\theta}(\mathbf{x}_i)$$

where ϕ_{θ} is a deep network with parameters θ .

Result: a **deep matrix factorization** model. The user/task side is still a small embedding $\mathbf{u}_m$; the item/input side is a learned non-linear feature extractor.

This is the architecture of every modern two-tower recommender system (e.g., YouTube, Spotify, Pinterest).

Same training: alternating optimization or SGD, only now θ is learned by backprop.

Non-Negative Matrix Factorization

Parts-based representations and topic models

When PCA Falls Short

Suppose all the data is non-negative. Examples:

- Pixel intensities (always ≥ 0).
- Word counts in a document.
- Spending in a category.
- Concentrations of chemical components.

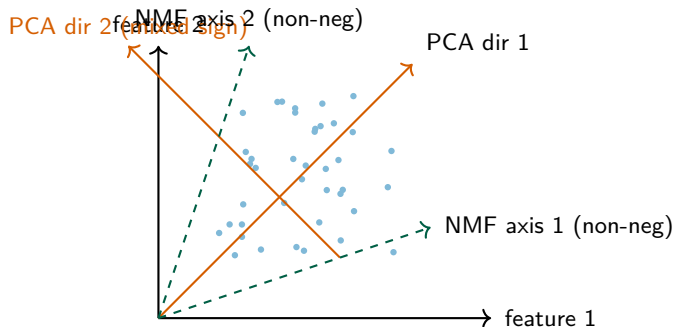
Issue with PCA: principal components can have *negative* entries – so reconstructions involve *subtracting* basis elements.

Why is this a problem?

- Hard to interpret: “a face is the average face minus 0.3 of the cheekbone direction”.
- Doesn't reflect how the world actually decomposes: documents are *combinations* of topics, not topics minus topics.

We want a parts-based decomposition: $\mathbf{Y} = \mathbf{UV}^T$ with $\mathbf{U}, \mathbf{V} \geq 0$.

Visual: PCA Subspace vs Non-Negative Subspace



PCA (red): orthogonal axes capturing maximum variance, but axis 2 has negative components.

NMF (green): non-orthogonal but non-negative axes. Each axis is a “part” that you *add* to build the data.

Different geometric prior, very different interpretation.

NMF: The Optimization Problem

Given: non-negative matrix $\mathbf{Y} \geq 0$ of size $M \times N$.

Find: non-negative $\mathbf{U} \in \mathbb{R}_+^{M \times K}$ and $\mathbf{V} \in \mathbb{R}_+^{N \times K}$ such that $\mathbf{Y} \approx \mathbf{UV}^\top$.

Two common loss functions:

Squared loss

$$\min_{\mathbf{U} \geq 0, \mathbf{V} \geq 0} \sum_{i,j} (Y_{ij} - \mathbf{u}_i^\top \mathbf{v}_j)^2$$

penalizes squared distance.

Generalized KL divergence (a.k.a. unnormalized KL)

$$\min_{\mathbf{U} \geq 0, \mathbf{V} \geq 0} \sum_{i,j} Y_{ij} \log \frac{Y_{ij}}{\mathbf{u}_i^\top \mathbf{v}_j} - Y_{ij} + \mathbf{u}_i^\top \mathbf{v}_j$$

penalizes ratio – more appropriate for count data.

Train with multiplicative updates (Lee & Seung, 2001) or projected gradient descent.

Faces: PCA vs NMF

Same dataset, two decompositions:

PCA (eigenfaces):

- Each basis face has both positive and negative pixels.
- Coefficients can be negative.
- Reconstructions look “ghostly” – holistic faces.
- Few basis elements give a good reconstruction.

NMF (non-negative basis):

- Each basis face is a localized “part” – nose region, eye region, jaw.
- Coefficients are non-negative – they say *how much* of each part to use.
- Reconstructions assemble parts additively.
- Need more basis elements for the same reconstruction quality.

Tradeoff: NMF is less compact but far easier to interpret.

Topic Modeling: NMF on Text

Setup: build a document-by-word *count* matrix $\mathbf{Y} \in \mathbb{R}_+^{M \times N}$.

Y_{ij} = number of times word j appears in document i .

Apply NMF with K topics:

- $\mathbf{U} \in \mathbb{R}_+^{M \times K}$: how much of each topic appears in each document.
- $\mathbf{V} \in \mathbb{R}_+^{N \times K}$: which words characterize each topic.

Interpretation:

- Sort each column of $\mathbf{V}$ by weight, top words define the topic.
- Sort each row of $\mathbf{U}$ to see the document's topic mixture.

LDA connection: Latent Dirichlet Allocation is the probabilistic cousin of NMF – same conceptual structure, different statistical assumptions.

Econ Applications of NMF / LDA on Text

Three landmark papers:

1. **Hansen, McMahon, Prat (2018, QJE).**

LDA on FOMC transcripts. Shows that committee deliberation patterns predict policy decisions and macro outcomes.

2. **Gentzkow, Shapiro, Taddy (2019, *Econometrica*).**

Text-as-data framework using bilinear / NMF-style decompositions. Recovers ideological positions of US politicians from speech.

3. **Bandiera, Prat, Hansen, Sadun (2020, JPE).**

Topic models on CEO time-use diaries. Recovers two CEO “types” that predict firm performance.

Pattern: reduce a high-dimensional, sparse, non-negative count matrix to a small set of interpretable factors. Then run regular econometrics on those factors.

SVD/PCA vs NMF: Side-by-Side

	SVD / PCA	NMF
Basis vectors	orthonormal, mixed sign	non-negative, non-orthogonal
Coefficients	any sign	non-negative
Geometric meaning	best rotation of axes	cone of additive parts
Interpretability	holistic, ghostly	parts-based, intuitive
Reconstruction quality	best at fixed K	needs more components
Best for	Gaussian-like data	non-negative / count data

Choose based on the data and the question. For interpretability with non-negative data, NMF wins. For pure compression, SVD wins.

Bridge to Deep Learning

What changes when we go non-linear?

PCA Is a Linear Autoencoder

Encoder-decoder view:

$$\mathbf{x} \xrightarrow{\text{encode}} \mathbf{z} = \mathbf{U}^\top \mathbf{x} \xrightarrow{\text{decode}} \hat{\mathbf{x}} = \mathbf{U} \mathbf{z}$$

$\mathbf{U}$ is the same on both sides – this is called **tied weights**.

Loss:

$$\min_{\mathbf{U}} \sum_i \|\mathbf{x}_i - \mathbf{U} \mathbf{U}^\top \mathbf{x}_i\|^2$$

Solution: the columns of $\mathbf{U}$ are the top- K left singular vectors. *This is exactly PCA.*

Generalization: replace the linear encoder $\mathbf{U}^\top$ with a deep network, the linear decoder $\mathbf{U}$ with another deep network, and untie the weights. You get a **deep autoencoder**.

The benefit: the bottleneck $\mathbf{z}$ can capture non-linear manifolds (faces with expression, lighting, pose) that PCA can't.

Embedding Layers Are Matrix Factorization

In PyTorch:

```
1 emb = nn.Embedding(num_users, K)
2 u_i = emb(user_id)  # K-dim vector
```

What this is doing: `emb.weight` is a $\text{num_users} \times K$ matrix. `emb(user_id)` returns row `user_id`. That's the user vector $\mathbf{u}_i$ from collaborative filtering.

Two embeddings + dot product = matrix factorization.

Where it matters:

- Recommender systems (user $\times$ item).
- Word embeddings (word $\times$ context word).
- Knowledge graph embeddings (entity $\times$ relation $\times$ entity).
- Two-tower retrieval models (query $\times$ document).

Whenever you see an embedding layer, you're looking at the **U** from this lecture.

Word2vec Is Implicit Matrix Factorization

Skip-gram with negative sampling (Mikolov et al., 2013):

- Slide a window over text.
- For each center word w and context word c in the window: predict “ w and c co-occur”.
- For randomly drawn negative pairs: predict “they don’t”.

Levy & Goldberg (NIPS 2014) theorem: this objective implicitly factorizes the matrix

$$M_{wc} = \text{PMI}(w, c) - \log k$$

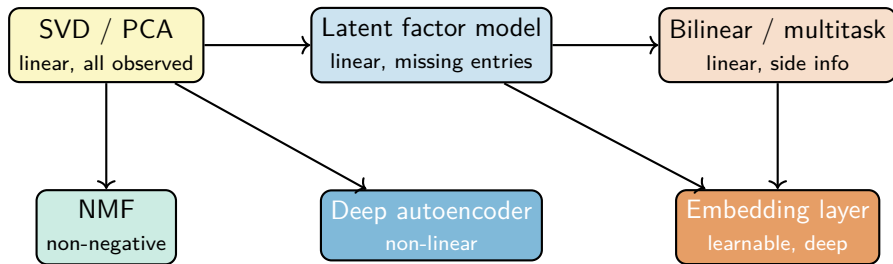
where PMI is pointwise mutual information and k is the negative-sampling rate.

Word2vec = SVD of the shifted PMI matrix.

Implications:

- Word embeddings are not magic – they are matrix factorization in disguise.
- “King – man + woman $\approx$ queen” is exactly a low-rank linear-algebra property.

What We've Built



One thread runs through everything: learn a low-dimensional representation that captures the structure of the data.

Linear bottleneck → non-linear bottleneck = the move from classical ML to deep learning.

Summary & Next Steps

Today's Big Ideas

1. **SVD is the master decomposition.** Best low-rank approximation in Frobenius norm; the foundation of PCA, eigenfaces, and everything that follows.
2. **Trace norm is the convex envelope of rank.** Replaces a discrete penalty with a smooth one – the matrix analogue of L1.
3. **Latent factor models** solve matrix completion with $\mathbf{Y} \approx \mathbf{UV}^\top$ via Frobenius regularization on the factors.
4. **Panel data view.** Same model as Bai (2009) interactive fixed effects; matrix completion of treated cells gives causal counterfactuals (Athey et al. 2021).
5. **NMF** replaces orthogonality with non-negativity. Yields parts-based, interpretable bases. Core technology for topic modeling.
6. **Bridge to DL.** Embedding layers, autoencoders, and word2vec are all matrix factorization with extra moving parts.

Practical Recipes

When to use what:

- Need to compress dense, mixed-sign data → **PCA / SVD**.
- Have a sparse user-by-item matrix → **latent factor models with ALS**.
- Causal inference with panel data and missing potential outcomes → **matrix completion** (gsynth, pysyncon).
- Topic modeling or non-negative parts → **NMF** (`sklearn.decomposition.NMF`).
- Deep features and large-scale learning → **embedding layers + neural net**.

Common bug: trying to apply SVD to a matrix with missing entries. SVD requires every cell – if you have missing data, use the latent factor formulation with the indicator ω_{ij} , not raw SVD.

Common bug: forgetting to standardize before PCA. PCA without centering puts the first component in the direction of the mean, not the maximum variance.

Next Lecture: Embeddings and Attention

Lecture 7:

- Embedding learning beyond linear factor models.
- Word2vec, item2vec, sentence embeddings.
- The attention mechanism: queries, keys, values.
- Self-attention as content-aware embedding lookup.
- Foundation for Transformers (Lecture 8).

The big picture:

SVD $\rightarrow$ Latent factors $\rightarrow$ Embeddings $\rightarrow$ Attention $\rightarrow$ Transformers

Recommended reading before next class:

- Mikolov et al. (2013), *Distributed representations of words and phrases*.
- Levy & Goldberg (2014), *Neural word embedding as implicit matrix factorization*.
- Athey et al. (2021) intro section, *Matrix completion methods for causal panel data*.

Questions?

References

- Eckart, C., & Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3).
- Lee, D. D., & Seung, H. S. (1999). Learning the parts of objects by non-negative matrix factorization. *Nature*, 401.
- Lee, D. D., & Seung, H. S. (2001). Algorithms for non-negative matrix factorization. *NIPS*.
- Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *IEEE Computer*, 42(8).
- Bai, J. (2009). Panel data models with interactive fixed effects. *Econometrica*, 77(4).
- Athey, S., Bayati, M., Doudchenko, N., Imbens, G., & Khosravi, K. (2021). Matrix completion methods for causal panel data models. *JASA*, 116(536).
- Xu, Y. (2017). Generalized synthetic control method. *Political Analysis*, 25(1).
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). Distributed representations of words and phrases. *NIPS*.
- Levy, O., & Goldberg, Y. (2014). Neural word embedding as implicit matrix factorization. *NIPS*.
- Hansen, S., McMahon, M., & Prat, A. (2018). Transparency and deliberation within the FOMC. *QJE*, 133(2).
- Gentzkow, M., Kelly, B., & Taddy, M. (2019). Text as data. *Journal of Economic Literature*, 57(3).
- Bandiera, O., Prat, A., Hansen, S., & Sadun, R. (2020). CEO behavior and firm performance. *JPE*, 128(4).